

RouteAI

Intelligent LLM Routing & Cost Optimization Platform

PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 • September 2026 • Status: Product Definition

Purpose: Define a production-oriented platform that gives developers a unified AI gateway and intelligently selects models based on cost, latency, quality, reliability, and explicit user constraints.

This PRD covers product scope, UX, routing algorithms, AI/ML systems, APIs, architecture, database design, security, observability, evaluation, roadmap, risks, and acceptance criteria.

Document Overview

Section	Contents
1	Executive summary, vision, value proposition
2	Goals, non-goals and success definition
3	Users, personas and use cases
4	MVP and complete feature scope
5	UX/UI screens and interaction requirements
6	Routing engine and optimization logic
7	Token, cost, latency and quality prediction
8	System architecture and technology stack
9	Gateway and management API specification
10	Database entities and retention
11	Security, privacy and reliability
12	Observability and operational metrics
13	Benchmarking and evaluation methodology
14	Implementation roadmap
15	Acceptance criteria
16	Risks and mitigations
17	Future product direction

1. Executive Summary

RouteAI is an intelligent AI infrastructure layer positioned between an application and multiple LLM providers. It exposes a simple, OpenAI-compatible interface while deciding which underlying model should process each request. The decision considers predicted token usage, expected cost, latency, quality, reliability, model capabilities and developer-defined constraints.

The central thesis is that model selection should be an optimization problem rather than a hard-coded configuration. Simple requests should not automatically consume premium models, while difficult requests should not be routed to an inexpensive model that fails quality requirements.

RouteAI combines a gateway, model registry, routing engine, benchmarking system, observability layer, semantic cache, fallback system and optimization analytics into one developer platform.

Positioning: RouteAI should not be presented as merely another unified LLM API. Its differentiating layer is measurement and optimization: predicting the cost, latency and likely quality of a request, selecting an eligible model, executing it, evaluating the outcome, and using telemetry to improve future decisions.

1.1 Product Vision

Make model choice largely invisible to application developers while making AI infrastructure measurable, explainable, controllable and cost-efficient.

1.2 Core Value Proposition

- One API for multiple AI providers and models.
- Automatic model selection for each request.
- Budget-, latency- and quality-aware routing.
- Transparent explanations for routing decisions.
- Automatic fallback for retryable provider failures.
- Token and cost forecasting before execution.
- Benchmark-driven model quality data.
- Semantic caching and prompt optimization.
- Analytics quantifying savings, latency, quality and reliability.
- Architecture suitable for a portfolio project and eventual SaaS product.

2. Goals & Non-Goals

2.1 Goals

- Provide a low-friction multi-model gateway.
- Reduce unnecessary AI spend without materially degrading quality.
- Improve reliability through provider/model fallback.
- Give developers hard constraints and optimization preferences.
- Create benchmark evidence for model selection.
- Create a feedback loop from production outcomes into routing.
- Start deterministic and introduce ML only after sufficient data exists.

2.2 Non-Goals

- Training a foundation model from scratch.
- Replacing every provider-specific capability.

- Guaranteeing predicted quality equals human judgment.
- Building a general-purpose observability platform.
- Starting with Kubernetes/Kafka before scale requires them.

3. Users & Use Cases

3.1 Target Users

Persona	Need	RouteAI Value
Indie developer	Ship AI features cheaply	Gateway + automatic routing
Startup engineer	Control growing AI spend	Budgets + optimization
AI engineer	Compare models	Playground + benchmarks
Platform engineer	Standardize AI access	Unified API + policies
Researcher	Measure trade-offs	Repeatable evaluation

3.2 Core Use Cases

- Automatic routing: model=auto selects an appropriate candidate.
- Hard constraints: maximum cost, latency, minimum quality and capability requirements.
- Fallback: retryable failures move to an eligible alternative.
- Cost analysis: actual spend is compared with baseline spend.
- Benchmarking: identical workloads are evaluated across models.
- Semantic caching: equivalent requests can reuse safe responses.
- Optimization: identify workloads likely to work on cheaper/faster models.

4. Product Scope

4.1 MVP

The MVP must prove the complete loop: request → analyze → select → execute → measure → explain. Support 3-5 models across at least two providers and expose an OpenAI-compatible API.

Feature	MVP Requirement	Priority
Unified Gateway	OpenAI-compatible chat endpoint	P0
Model Registry	Pricing, context, capabilities, health	P0
Routing Engine	Constraint-aware deterministic scoring	P0
Usage Tracking	Tokens, latency, status, model/provider	P0
Dashboard	Cost, requests, latency, distribution	P0
Request Explorer	Request + candidates + reason	P0
Fallback	Configurable alternatives	P0
API Keys	Project keys and revocation	P0
Token Estimate	Pre-request estimate	P1
Benchmarking	Small versioned suite	P1

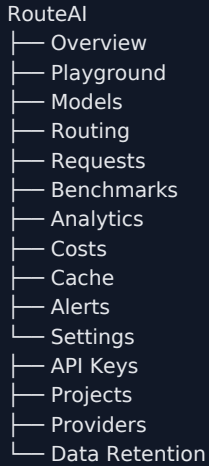
4.2 Full Product

Area	Capabilities
Routing	Constraint solver, adaptive routing, policies, provider/model selection
Prediction	Token, cost, latency, quality, failure probability
Reliability	Fallback chains, retries, circuit breakers, health scoring
Optimization	Semantic cache, prompt compression experiments
Evaluation	LLM-as-judge, deterministic checks, benchmark datasets
Analytics	Cost, savings, latency percentiles, quality, reliability
Developer Tools	Playground, SDKs, keys, webhooks, logs, replay
Governance	Budgets, alerts, rate limits, retention, policies
ML	Learned routing model and contextual-bandit research

5. UX / UI Specification

The dashboard should feel like developer infrastructure software: information-dense, explainable and operational. A dark interface is suitable for the primary experience, with restrained status indicators.

5.1 Navigation



```
RouteAI
├─ Overview
├─ Playground
├─ Models
├─ Routing
├─ Requests
├─ Benchmarks
├─ Analytics
├─ Costs
├─ Cache
├─ Alerts
├─ Settings
├─ API Keys
├─ Projects
├─ Providers
└─ Data Retention
```

5.2 Overview Dashboard

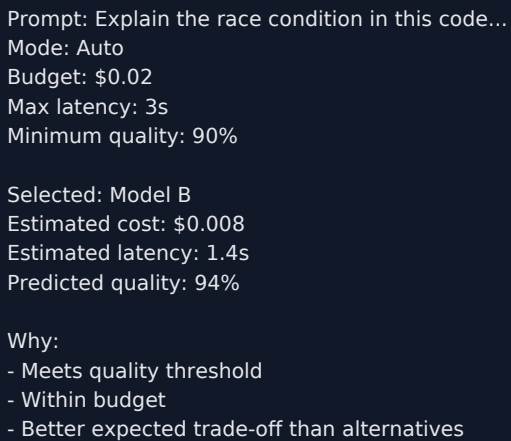
The overview should immediately answer: how much was spent, how much was saved, which models are being used, whether latency is changing, and whether providers are failing.

Key widgets:

- Total requests and success/failure rate.
- Actual spend versus baseline spend.
- Estimated savings percentage and absolute savings.
- Average, P50, P95 and P99 latency.
- Model/provider distribution.
- Cache hit rate and saved cost.
- Recent alerts and optimization recommendations.
- Cost and latency trend charts.

5.3 Playground

A developer enters a prompt and sets budget, latency and quality preferences. The UI displays the selected model, candidate comparison, estimates, actual results and explanation.



```
Prompt: Explain the race condition in this code...
Mode: Auto
Budget: $0.02
Max latency: 3s
Minimum quality: 90%

Selected: Model B
Estimated cost: $0.008
Estimated latency: 1.4s
Predicted quality: 94%

Why:
- Meets quality threshold
- Within budget
- Better expected trade-off than alternatives
```

5.4 Request Explorer

- Request ID and timestamp.
- Application/project.
- Input/output/total tokens.
- Candidate models and predicted scores.
- Selected model and fallback path.
- Actual cost and latency.
- Evaluation score when enabled.
- Provider response status and normalized error.
- Routing explanation and applied policy.

5.5 Models Screen

Each model should show price, context window, capabilities, measured latency, availability, benchmark quality and recent error rate. Clearly distinguish provider-published metadata from RouteAI-measured production metrics.

6. Core Routing Engine

6.1 Routing Pipeline

```
Incoming request
↓
Authentication + project policy
↓
Request normalization
↓
Feature extraction
↓
Token estimation
↓
Candidate filtering
↓
Cost / latency / quality prediction
↓
Constraint validation
↓
Optimization / scoring
↓
Model + provider selection
↓
Execution
↓
Evaluation + telemetry
↓
Feedback store
```

6.2 Request Features

- Estimated input tokens and maximum output tokens.
- Task category: coding, extraction, summarization, reasoning, classification, etc.
- Prompt complexity and context size.
- Required capabilities: vision, tools, JSON/structured output.
- Developer policy and preferred providers.
- Historical workload behavior.
- Provider/model health.

6.3 Hard Candidate Filtering

Filtering occurs before scoring. A model that cannot satisfy a hard requirement must not win because it is cheap.

Examples:

- Context window $\geq$ estimated context
- Vision required $\rightarrow$ model supports vision
- Structured output required $\rightarrow$ capability exists
- Predicted/known cost $\leq$ hard budget
- Provider/model allowed by policy

6.4 Baseline Score

score =

$W_q * \text{quality}$
 $+ W_r * \text{reliability}$
 $+ W_s * \text{speed}$
 $- W_c * \text{cost}$
 $- W_l * \text{latency}$

subject to:

$\text{cost} \leq \text{max_cost}$
 $\text{quality} \geq \text{min_quality}$
 $\text{required_capabilities} \subseteq \text{model_capabilities}$

The deterministic scorer should be modular so it can later be replaced by a learned policy.

6.5 Routing Policies

Policy	Behavior
Cheapest	Minimize expected cost subject to constraints
Fastest	Minimize expected latency subject to constraints
Quality First	Maximize predicted quality subject to budget
Balanced	Weighted cost + latency + quality
Custom	Developer-defined weights and hard constraints

7. AI / ML Systems

ML is introduced progressively. MVP routing can use provider metadata and measured statistics; learned prediction becomes valuable after sufficient benchmark and production telemetry exists.

7.1 Token Prediction

Use provider/model tokenizers where available. Estimate input tokens before execution and predict output-token distributions using prompt length, task type and historical workloads.

7.2 Cost Prediction

```
estimated_cost =  
input_tokens × current_input_price  
+ output_tokens × current_output_price
```

Actual provider usage becomes ground truth.

Pricing records must be versioned with effective timestamps so historical requests remain reproducible.

7.3 Latency Prediction

Track time-to-first-token, total latency and tokens/second for streaming. Predict latency using model/provider, token counts, time-of-day, region and historical workload features. Use percentiles, not only averages.

7.4 Quality Prediction

Quality prediction is the hardest component and must be treated probabilistically. Maintain versioned benchmark tasks with deterministic graders where possible and LLM/human evaluation for open-ended tasks.

- JSON/schema validity.
- Exact/semantic match for extraction and classification.
- Safe code execution tests where appropriate.
- LLM-as-judge with calibrated rubrics.
- Human samples for evaluator calibration.
- Confidence intervals and evaluator agreement.

7.5 Learned Routing

```
request features  
↓  
candidate outcome predictions  
↓  
P(quality >= threshold)  
expected_cost  
expected_latency  
failure_probability  
↓  
constraint optimizer  
↓  
selected candidate
```

7.6 Contextual Bandit — Future

A contextual bandit can cautiously explore alternative models while optimizing long-term reward. Exploration must obey budgets, quality floors and provider policies. This is a post-MVP research feature.

8. Platform Architecture

8.1 Recommended Stack

Layer	Technology
Frontend	React + TypeScript + Vite + Tailwind
API	Python + FastAPI + Pydantic
Database	PostgreSQL
Vector Search	pgvector
Cache	Redis
Background Jobs	Celery/RQ initially
Provider Adapters	Common interface over provider SDKs
Observability	OpenTelemetry + Prometheus/Grafana
Deployment	Docker + reverse proxy + CI/CD
Testing	Pytest + integration + benchmark harness

8.2 Logical Architecture

```
React Dashboard
↓
FastAPI Control API
↓
Gateway / Auth / Analytics
↓
Routing Engine
├─ Token & Cost Prediction
├─ Quality / Latency Prediction
├─ Cache
├─ Health
↓
Provider Adapters
├─ Provider A
├─ Provider B
├─ Provider C
```

8.3 Provider Adapter Interface

```
class ModelProvider:
    list_models()
    estimate_cost(request, model)
    chat(request, model)
    stream(request, model)
    health_check(model)
    normalize_error(error)
```

9. API Specification

9.1 Authentication

Project API keys must be hashed at rest, displayed only at creation, revocable and scoped to a project. Provider master keys must never reach the browser.

9.2 Gateway Endpoint

```
POST /v1/chat/completions
```

```
{
  "model": "auto",
  "messages": [
    {"role": "user", "content": "Explain this error..."}
  ],
  "route": {
    "budget_usd": 0.02,
    "max_latency_ms": 3000,
    "min_quality": 0.90,
    "policy": "balanced"
  }
}
```

9.3 Response Metadata

```
{
  "id": "req_98122",
  "model": "model-b",
  "provider": "provider-x",
  "usage": {
    "input_tokens": 382,
    "output_tokens": 216,
    "total_tokens": 598
  },
  "routing": {
    "estimated_cost": 0.0042,
    "predicted_quality": 0.94,
    "predicted_latency_ms": 1400,
    "fallback_used": false
  }
}
```

9.4 Management Endpoints

Endpoint	Purpose
GET /v1/models	Available models and capabilities
GET /v1/requests	Paginated request history
GET /v1/requests/{id}	Request and routing details
GET /v1/analytics/cost	Cost and savings analytics
GET /v1/analytics/latency	Latency percentiles and trends
POST /v1/benchmarks/run	Run benchmark suite
GET /v1/benchmarks/{id}	Benchmark results
POST /v1/cache/invalidate	Invalidate cache entries
GET /v1/alerts	List alerts
POST /v1/api-keys	Create project key

10. Data Model

10.1 Core Entities

Entity	Important Fields
users	id, email/SSO identity, created_at
projects	id, user_id, name, policy_id, retention
api_keys	id, project_id, key_hash, prefix, revoked_at
models	id, provider_id, name, capabilities, context_window
model_prices	model_id, prices, effective_from, effective_to
model_metrics	model_id, window, latency, TTFT, error_rate, quality
requests	id, project_id, model, provider, timestamps, status, token counts
routing_decisions	request_id, candidates, scores, constraints, selected
evaluations	request_id, evaluator, dimensions, score, confidence
benchmarks	id, dataset, version, status
benchmark_results	benchmark_id, model_id, task, score, cost, latency
cache_entries	id, project_id, embedding, response, expires_at
budgets	project_id, period, limit, current_spend
alerts	project_id, type, threshold, status, triggered_at

10.2 Retention

- Store request metadata longer than raw prompt/response content.
- Allow project-level retention periods.
- Offer metadata-only logging mode.
- Never log API keys or provider secrets.
- Encrypt sensitive stored data and use least-privilege database access.

11. Security, Privacy & Reliability

11.1 Security Requirements

- TLS for all communication.
- Hash RouteAI API keys.
- Encrypt provider credentials at rest.
- Strong project-level authorization boundaries.
- Gateway rate limiting and request-size limits.
- Sanitize logs to prevent prompt/secret leakage.
- Audit security-sensitive actions.

11.2 Failure Handling

```
Request
↓
Primary model
├─ success → return
├─ rate limit → fallback
```

- └ timeout → fallback if allowed
- └ provider 5xx → fallback
- └ invalid request → do not blindly retry

Use exponential backoff with jitter for retryable errors, circuit breakers for persistently unhealthy providers, and a maximum retry/fallback budget.

11.3 Initial Reliability Targets

Metric	Target
Gateway availability	99.5% MVP target
Routing overhead	<100 ms target excluding provider latency
Eligible fallback recovery	>95% target
Analytics freshness	<5 minutes
Auditability	Every decision has immutable request ID

12. Observability & Operations

Every request should produce structured telemetry because the product promise depends on proving cost, latency, quality and reliability improvements.

Category	Metrics
Traffic	Requests/sec, volume, success rate
Cost	Actual cost, estimated cost, savings
Latency	TTFT, total latency, tokens/sec, P50/P95/P99
Routing	Selection frequency, policy distribution, constraint failures
Reliability	Provider error, timeout, fallback rate
Quality	Evaluation score, benchmark score, agreement
Cache	Hit rate, saved tokens, saved cost

Example Alerts

- Provider error rate >5% over 5 minutes.
- P95 latency exceeds policy threshold.
- Project spends >80% of budget.
- Unexpected cost >2× rolling baseline.
- Cache hit rate drops materially.
- Quality falls below configured threshold.

13. Evaluation & Benchmarking

The router must be evaluated as a system. Lower cost is not useful if outcomes become materially worse; higher quality is not useful if cost increases disproportionately.

13.1 Benchmark Categories

- Reasoning/math
- Coding/debugging
- Summarization
- Information extraction
- Classification
- SQL generation
- Structured JSON
- Long-context tasks
- Instruction following

13.2 Required Baselines

Baseline	Purpose
Premium model only	Quality/expense baseline
Cheap model only	Low-cost baseline
Fixed balanced model	Practical single-model baseline
Rule-based router	Pre-ML routing baseline
RouteAI	Final optimization policy

13.3 Primary Metrics

- Quality retention versus premium baseline.
- Cost reduction versus premium baseline.
- Latency reduction.
- Failure and fallback rates.
- Routing regret versus best eligible observed candidate.
- Cache-adjusted savings.
- Calibration error for cost/latency/quality predictions.

14. Implementation Roadmap

Phase	Deliverables	Outcome
0 — Foundation	Repo, Docker, DB, auth skeleton, adapter interface	Development platform
1 — MVP Gateway	Endpoint, 3-5 models, registry, routing	End-to-end path
2 — Dashboard	Overview, requests, models, analytics	Visible product
3 — Reliability	Fallback, retries, health, circuit breaker	Resilient gateway
4 — Prediction	Token/cost/latency estimation	Pre-request optimization
5 — Evaluation	Benchmark harness + quality evaluation	Evidence-based routing
6 — Optimization	Semantic cache + compression experiments	Additional savings

Phase	Deliverables	Outcome
7 — Learned Router	Prediction model + constrained optimizer	Adaptive routing
8 — SaaS Hardening	Teams, quotas, metering, docs	Productization

Recommended Order

Gateway → registry → deterministic router → telemetry → dashboard → fallback → prediction → evaluation → cache → learned routing. Do not begin with Kubernetes, Kafka or a complex ML pipeline.

15. Acceptance Criteria

Area	Definition of Done
Gateway	OpenAI-compatible request returns normalized response.
Routing	Eligible candidates are scored and selection is recorded.
Constraints	Hard capability/policy constraints cannot be violated.
Telemetry	Tokens, latency, cost estimate, actual cost, model, provider, status and request ID recorded.
Fallback	Retryable failures can use permitted alternatives.
Dashboard	Requests, cost, savings, latency and model distribution visible.
Explainability	Decision exposes candidates, constraints and reason.
Benchmarking	Repeatable versioned benchmark compares at least three models.
Security	API keys are never logged or stored plaintext.
Testing	Critical routing/failure paths covered by automated tests.

16. Risks & Mitigations

Risk	Impact	Mitigation
Pricing changes	Bad cost prediction	Versioned prices + refresh
Quality prediction errors	Bad routing	Benchmarks + confidence + quality floor
Provider differences	Integration complexity	Adapter interface
Router latency	Poor UX	Fast scoring + cached metadata
Bad cache match	Incorrect response	Threshold + namespace + TTL + opt-in
Sensitive prompts	Privacy risk	Minimal retention + encryption + redaction
Availability changes	Failed routing	Health checks + dynamic filtering
Over-optimization	Quality degradation	Quality floor + A/B testing + rollback

17. Future Product Direction

After the core routing loop is reliable, RouteAI can evolve into a broader AI optimization platform.

- Automatic policy learning from workload outcomes.
- Workload-specific routing policies for coding, support, extraction and reasoning.
- Provider arbitrage across equivalent capabilities.
- Hard monthly budgets, quotas and emergency policies.
- Regression detection after model/provider changes.
- SDKs for Python, TypeScript, Java and Go.
- Optional open-source routing/benchmark core with hosted analytics.

17.1 Simulation Mode

Developers provide monthly request volume, token distributions, quality targets and latency requirements. RouteAI simulates policies before deployment.

1,000,000 requests / month
Average input: 1,200 tokens

Average output: 500 tokens

Premium only: \$1,840/month

Cheap only: \$410/month

RouteAI: \$680/month

Estimated savings vs premium: 63%

Quality target: 94%

P95 latency: 2.1s

17.2 Final Product Definition

RouteAI succeeds when developers no longer need to manually choose an LLM for every request. Developers define acceptable cost, latency, quality and capability constraints; RouteAI handles selection, execution, fallback, measurement and continuous optimization.

Appendix A — Suggested Repository Structure

```
routeai/  
├── apps/  
│   ├── api/  
│   └── dashboard/  
├── packages/  
│   ├── routing/  
│   ├── providers/  
│   ├── tokenization/  
│   ├── evaluation/  
│   └── common/  
├── services/  
│   ├── worker/  
│   └── scheduler/  
├── infra/  
├── benchmarks/  
├── migrations/  
├── tests/  
└── docs/
```

Appendix B — MVP Demo Script

- Create project and API key.
- Send simple request with model=auto.
- Show low-cost model selection.
- Send complex reasoning request and show stronger model.
- Open Request Explorer and explain candidates.
- Simulate provider failure and demonstrate fallback.
- Show cost, latency, model distribution and savings.
- Run benchmark comparison.
- Change policy from Cheapest to Quality First.
- Run Simulation Mode.

Appendix C — Portfolio Positioning

Present RouteAI as an AI infrastructure and optimization system, not simply an LLM wrapper.

RouteAI — Intelligent LLM Routing & Cost Optimization Platform

Built an OpenAI-compatible multi-provider AI gateway that dynamically selects models using cost, latency, quality, capability and reliability constraints; implemented benchmarking, automatic fallback, token/cost prediction, semantic caching, evaluation and production analytics.

Only add numerical performance claims after benchmarking the implementation. Useful evidence includes measured cost reduction, routing overhead, quality retention, cache hit rate and fallback recovery rate.